Applied Data Preparation Lab

Healthcare Dataset Preprocessing Pipeline

Vijay Kota

1. Introduction

Data preparation is one of the most important stages of a machine learning workflow. Real-world datasets are rarely ready to be directly used by machine learning algorithms. They may contain missing values, inconsistent categorical representations, numerical variables with different scales, and columns that should not be used as predictive features.

This assignment focuses on preparing a real-world healthcare dataset for future machine learning applications.

The selected dataset is the:

Diabetes 130-US Hospitals for Years 1999–2008

The dataset contains hospital records of diabetic patients and includes information related to admission, diagnosis, laboratory procedures, medications, demographic characteristics, and readmission status.

The objective of this assignment is to construct a complete and reproducible preprocessing pipeline that:

- Identifies and handles missing values.
- Processes categorical variables.
- Encodes categorical features into numerical representations.
- Scales numerical features.
- Removes inappropriate identifier columns.
- Produces a clean dataset suitable for future machine learning workflows.

No machine learning model is trained in this assignment. The focus is entirely on data preparation.

2. Objectives

The main objectives of this assignment are:

1. Load and inspect a real-world healthcare dataset.
2. Identify missing values and understand their representation.
3. Separate numerical and categorical features.
4. Remove non-predictive identifier columns.

5. Handle missing numerical values using an appropriate strategy.
6. Handle missing categorical values while preserving useful information.
7. Encode categorical variables using One-Hot Encoding.
8. Scale numerical variables using StandardScaler.
9. Build a reproducible preprocessing pipeline using Scikit-learn.
10. Compare the dataset before and after preprocessing.

3. Dataset Description

The dataset used in this assignment is the **Diabetes 130-US Hospitals for Years 1999–2008** dataset.

The dataset contains approximately 100,000 hospital encounters and includes 50 columns in its original form.

The actual dataset used in this project has:

101,766 rows and 50 columns

The dataset contains information about:

- Patient demographics
- Hospital admission information
- Diagnosis information
- Laboratory procedures
- Medication information
- Hospital stay duration
- Previous hospital visits
- Readmission status

The target variable in the original dataset is:

`readmitted`

However, the target variable is not included as an input feature during the preprocessing process because doing so would cause data leakage.

4. Initial Data Inspection

The original dataset was inspected using Pandas before any preprocessing was performed.

The original dimensions were:

$$101766 \times 50$$

The dataset contained both numerical and categorical variables.

An important characteristic of the dataset was the representation of missing values.

Instead of using standard Pandas `NaN` values, missing values were represented using the character:

`?`

These values were converted to proper missing values before applying the preprocessing pipeline.

5. Missing Value Analysis

Missing data is a major issue in the selected healthcare dataset.

After converting the `?` values into missing values, several columns were found to contain substantial amounts of missing information.

Important findings include:

Feature	**Approximate Missing Rate**
`weight`	96.8%
`medical_specialty`	49.0%
`payer_code`	39.5%

The high percentage of missing values makes it inappropriate to simply remove every row containing missing information.

Removing all incomplete rows could result in a significant loss of healthcare records and potentially introduce selection bias.

Therefore, an imputation-based strategy was selected.

6. Handling Missing Numerical Values

Missing numerical values were handled using **median imputation**.

The median of a numerical feature is calculated as:

$$Median(X)$$

and missing values are replaced by this value.

The reason for selecting median imputation is that the median is more robust to extreme values and statistical outliers than the mean.

For example, if a numerical healthcare variable contains a small number of unusually high values, replacing missing values with the mean could shift the imputed value toward those extreme observations.

Median imputation provides a more robust central estimate.

The Scikit-learn strategy used was:

```
SimpleImputer(strategy="median")
```

This process ensures that numerical columns contain valid numerical values before scaling.

7. Handling Missing Categorical Values

Categorical missing values were handled using a constant:

`Unknown`

Rather than removing records with missing categories, the missing value was explicitly represented as its own category.

This approach was selected because missingness itself can potentially contain useful information.

For example, the absence of a recorded medical specialty may be different from the presence of a known specialty.

Replacing missing categorical values with `Unknown` preserves the record while making the data compatible with machine learning algorithms.

The preprocessing strategy therefore uses:

```
SimpleImputer(strategy="constant", fill_value="Unknown")
```

8. Categorical Feature Processing

The dataset contains numerous categorical variables.

Examples include:

- `race`
- `gender`
- `age`
- `weight`
- `payer_code`
- `medical_specialty`
- `admission_type_id`
- `discharge_disposition_id`
- `admission_source_id`

A total of **39 categorical features** were identified by the preprocessing pipeline.

Machine learning algorithms generally require numerical input, so these categorical features need to be converted into numerical representations.

9. Categorical Encoding

Categorical variables were encoded using **One-Hot Encoding**.

The Scikit-learn implementation used:

```
OneHotEncoder(handle_unknown="ignore")
```

For example, a categorical variable:

`gender`

could contain:

- Male
- Female

One-Hot Encoding represents these categories using separate binary features rather than assigning arbitrary numerical values.

This is important because assigning:

$$Male = 0$$

and:

$$Female = 1$$

could incorrectly suggest that Female is mathematically greater than Male.

One-Hot Encoding avoids introducing this artificial ordering.

The option:

```
handle_unknown="ignore"
```

was also selected so that previously unseen categories do not cause the preprocessing pipeline to fail when new data is processed.

10. Numerical Feature Processing

The preprocessing pipeline identified eight numerical input features:

- `time_in_hospital`
- `num_lab_procedures`
- `num_procedures`
- `num_medications`
- `number_outpatient`
- `number_emergency`
- `number_inpatient`
- `number_diagnoses`

These variables have different numerical ranges.

For example, the number of laboratory procedures and the length of hospital stay do not naturally exist on the same scale.

Without scaling, some algorithms may give disproportionate importance to features simply because their numerical values have a larger magnitude.

11. Numerical Feature Scaling

Numerical features were scaled using:

StandardScaler

The StandardScaler transformation is:

$$z = \frac{x - \mu}{\sigma}$$

where:

- x is the original value.
- μ is the mean of the feature.
- σ is the standard deviation.
- z is the standardized value.

After standardization, numerical features have approximately:

$$\mu \approx 0$$

and:

$$\sigma \approx 1$$

This makes numerical features more comparable and is especially useful for algorithms that are sensitive to feature scale.

12. Feature Exclusion

Two identifier columns were excluded from the input feature set:

- `encounter_id`
- `patient_nbr`

These columns identify individual records or patients rather than representing meaningful medical characteristics.

Including such identifiers could cause a machine learning model to memorize individual records rather than learn meaningful relationships between medical features and outcomes.

Therefore, these columns were excluded before preprocessing.

The target variable:

`readmitted`

was also excluded from the input feature matrix.

This prevents target leakage because the target must not be used to construct the input features.

13. Complete Preprocessing Pipeline

A complete preprocessing pipeline was implemented using Scikit-learn's `ColumnTransformer` and `Pipeline`.

The numerical processing flow is:

$$\texttt{Numerical Features} \rightarrow \texttt{Median Imputation} \rightarrow \texttt{StandardScaler}$$

The categorical processing flow is:

$$\texttt{Categorical Features} \rightarrow \texttt{Unknown Imputation} \rightarrow \texttt{OneHotEncoder}$$

These two pipelines are combined using:

$$\texttt{ColumnTransformer}$$

Conceptually:

```
Numerical Pipeline:
    SimpleImputer(strategy="median")
        |
    StandardScaler()

Categorical Pipeline:
    SimpleImputer(strategy="constant",
                  fill_value="Unknown")
        |
    OneHotEncoder(handle_unknown="ignore")

Combined using:
    ColumnTransformer
```

This approach makes the preprocessing process reproducible and reduces the possibility of accidentally applying different transformations to different datasets.

14. Before Preprocessing

Before preprocessing, the dataset had:

$$\textbf{101,766 rows} \times \textbf{50 columns}$$

The input feature set contained:

$$\textbf{47 features}$$

after excluding the two identifier columns and the target variable.

The original data contained:

- Missing values represented by `?`.
- Categorical variables represented as text/categories.

- Numerical variables with different scales.
- Identifier columns.
- A target variable that should not be used as an input feature.

A representative snapshot of the raw data was saved as:

`before_snapshot.csv`

This file demonstrates the state of the dataset before preprocessing.

15. After Preprocessing

After applying the preprocessing pipeline, the resulting feature matrix contained:

2,518 processed features

The increase from 47 input features to 2,518 processed features is primarily caused by One-Hot Encoding.

A single categorical feature may contain many different categories. One-Hot Encoding converts these categories into multiple binary indicator columns.

The final processed dataset contains:

- No remaining missing values.
- Numerical features scaled using StandardScaler.
- Categorical features converted into numerical indicators.
- Identifier columns excluded.
- Target variable excluded from input features.

A representative snapshot of the processed data was saved as:

`after_snapshot.csv`

16. Before and After Comparison

Before Preprocessing	**After Preprocessing**
47 input features	2,518 processed features
Missing values present	0 remaining missing values
Categorical values	One-Hot encoded numerical indicators
Different numerical scales	Standardized numerical features
Identifier columns present in raw data	Identifiers excluded from feature matrix
Target variable available	Target excluded from input features

The increase in dimensionality is expected because categorical features were expanded into multiple binary columns.

17. Preprocessing Verification

The preprocessing pipeline was executed successfully.

The following conditions were verified:

- Missing values remaining after preprocessing: **0**.
- Numerical features were scaled.
- Categorical features were encoded.
- Identifier columns were excluded.
- The target variable was excluded from the input features.
- The preprocessing pipeline executed without errors.

The script was executed using:

```
python3 data_preparation.py
```

The successful execution confirms that the complete preprocessing workflow is functional.

18. Justification for Using a Pipeline

A Scikit-learn preprocessing pipeline was used instead of manually performing transformations on individual columns.

There are several reasons for this approach.

18.1 Reproducibility

The same preprocessing operations can be consistently applied to future data.

18.2 Consistency

Every numerical feature receives the same numerical preprocessing strategy, while categorical features receive the same categorical processing strategy.

18.3 Reduced Errors

Combining preprocessing operations into a pipeline reduces the risk of forgetting a transformation or applying it incorrectly.

18.4 Future Machine Learning Integration

The preprocessing pipeline can later be connected directly to a machine-learning estimator.

This makes the workflow suitable for future predictive modeling tasks.

19. Healthcare Data Considerations

Healthcare data requires additional care during preprocessing.

Missing values should not automatically be treated as errors. In some cases, the absence of information can itself contain useful information.

For this reason, categorical missing values were represented as `Unknown` rather than simply deleting the corresponding records.

Similarly, removing every row containing missing information could significantly reduce the dataset and potentially introduce bias.

Identifier variables were also excluded because using them as predictive features could result in memorization rather than meaningful learning.

The preprocessing decisions were therefore made with both technical and healthcare data considerations in mind.

20. Limitations

Although the preprocessing pipeline provides a strong foundation for machine learning, several limitations remain.

1. Median imputation assumes that the median provides a reasonable estimate for missing numerical values.
2. Representing categorical missing values as `Unknown` preserves missingness but does not determine why the information was missing.
3. One-Hot Encoding significantly increases the dimensionality of the dataset, resulting in 2,518 processed features.
4. StandardScaler does not eliminate the influence of extreme observations completely.
5. No machine learning model was trained, so the effect of these preprocessing decisions on predictive performance has not yet been evaluated.

These limitations can be investigated in later machine learning assignments.

21. Project Structure

The final project is organized as follows:

```
Healthcare_Data_Preparation/
|
|-- dataset/
|   |
|   |-- diabetic_data.csv
|
|-- data_preparation.py
|
|-- before_snapshot.csv
|
|-- after_snapshot.csv
|
```

```
|-- preprocessing_report.md
```

The purpose of each file is:

File	Purpose
`diabetic_data.csv`	Original raw healthcare dataset
`data_preparation.py`	Main preprocessing implementation
`before_snapshot.csv`	Representative data before preprocessing
`after_snapshot.csv`	Representative data after preprocessing
`preprocessing_report.md`	Written explanation of preprocessing decisions

22. Assignment Requirement Mapping

The implementation satisfies the assignment requirements as follows:

Requirement	Implementation	Marks
Correct handling of missing values	Converted `?` to missing values and used median imputation for numerical features and `Unknown` for categorical features	3
Categorical variable encoding	One-Hot Encoding with `handle_unknown="ignore"`	2
Feature scaling	StandardScaler applied to numerical features	2
Quality of justification	Documented reasoning for missing values, encoding, scaling, identifiers, and healthcare considerations	3
Total		**10**

23. Screenshot Evidence

Two types of evidence should be included with the submission.

23.1 Before and After Dataset Evidence

The submitted snapshots should demonstrate the difference between the raw and processed data.

The following files were generated:

```
before_snapshot.csv
after_snapshot.csv
```

The before snapshot demonstrates the original messy state, while the after snapshot demonstrates the processed numerical feature matrix.

23.2 Execution Evidence

The preprocessing script can be executed using:

```
python3 data_preparation.py
```

A terminal screenshot should be captured showing the actual execution results and verification output.

24. Conclusion

This assignment successfully demonstrated the complete preparation of a real-world healthcare dataset for future machine learning applications.

The original dataset contained 101,766 records and 50 columns. After identifying the target and non-predictive identifiers, 47 input features were prepared.

Missing values represented by `?` were converted into proper missing values. Numerical missing values were handled using median imputation, while categorical missing values were represented using an `Unknown` category.

Categorical variables were converted into numerical representations using One-Hot Encoding. Numerical features were standardized using StandardScaler.

The complete preprocessing workflow was implemented using Scikit-learn's `Pipeline` and `ColumnTransformer`, producing a final processed feature space of 2,518 features.

The pipeline completed successfully with zero remaining missing values.

Overall, the assignment demonstrates the importance of careful data preparation before applying machine learning algorithms. The resulting pipeline provides a reproducible foundation that can be used in future assignments involving classification, regression, or other machine learning tasks.

— End of Report —